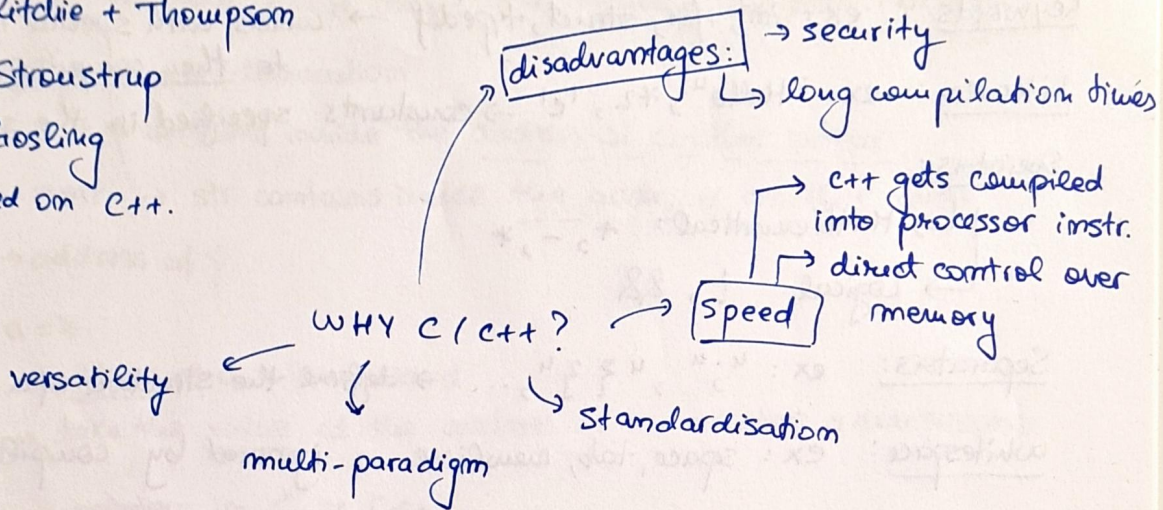


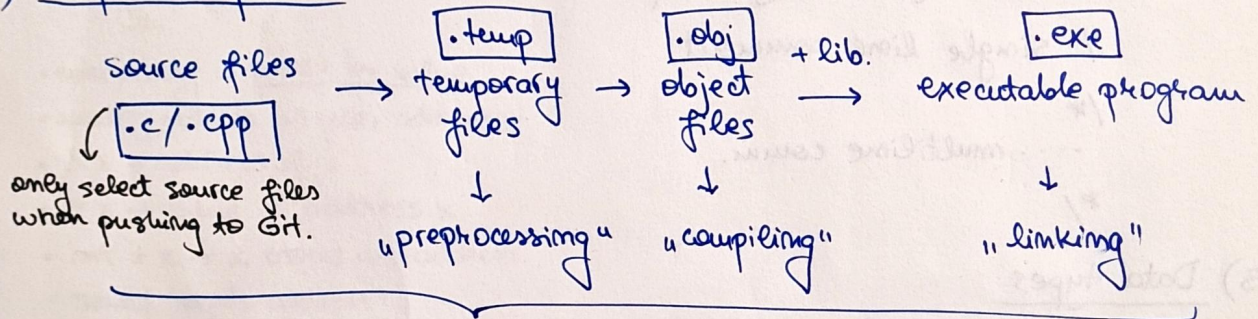
UNIX: Ritchie + Thompson

C++: Stroustrup

Java: Gosling
↳ based on C++.



1) Compilation process:



these steps happen before starting running the program.
(this is why C is fast).

! Source files are not enough to run the program.

! Advantage of a compiled language → SPEED.

- code translated to machine code before running
- no runtime interpretation
- optimizations made at compile.

```
#include <stdio.h>
#include <iostream>
```

→ preprocessor directives

`main` → function called by the OS.

Breakpoints → stop the program when reaching them.

COMPILER translates source code into executable code.

2)

Identifier : ex: i, MyFunction, res, _nameOfVariable

→ things that are built into the language

Keywords : ex: int, for, struct, typedef → words with special meaning to the compiler.

Literals : ex: "Hello", 72, 'c' → constants specified in the source code.

Operators:

↳ Mathematical: +, -, *

↳ Logical: !, &&

Separators: ex: ";", "{", "}", "... → define the structure of a program.

Whitespace: ex: space, tab, newline → ignored by compiler.

Comments: ignored by compiler.

// single line comment

/*

... multiline comm.

*/

3) Data types

TYPE = a domain of values + a set of operations defined on these values.

- char → 1 byte
- int → 4 bytes
- unsigned int → 4
- long int / long → 4
- float → 4
- double → 8
- long double → 8

Casting?

care the

Variable = named memory location.

pointer = a variable whose name is a memory location. (address)

int *p → pointer declaration

→ p contains inside the address of another integer.

char *str → str contains inside the addr. of another char

&p → address of p.

int a = 7

&a → address of a → ampersand

*a → take the value at the address of the pointer → dereference.

4) Behaviour of pointers in C vs C++:

C

- everything is passed by value
- modify outside ⇒ pass address.
- &x → address of x
- *x → value at address x
- int *x → x stores an address.
- scanf needs address!

ex: int x = 5

int *p = &x

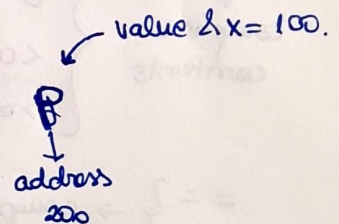
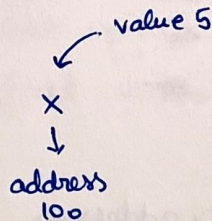
x ← address 100

p ← address 200

inside p ← store 100.
(address 200)

address 100 ← 5

int *p ← garbage.



& → address

* → value

5) Array = continuous block of memory.

- indexing $0, n-1$
- multidimensional array: $\text{mat}[m][m]$.
- the addresses of the elem. in array are consecutive nr.

Name of array = a pointer to the first elem. in the array.
(address of $b[0]$ = address of b).

Char arrays $\rightarrow$ last character is $'\backslash 0'$.

$b[3] \rightarrow$ value found at address $b+3$ is $\ast(b+3)$...

"
 $\ast(b+3)$

"
 $3[b]$

6) String manipulation (library (string.h)) ?

string = array of chars (kinda) ?

Strings in C $\rightarrow$ only "characters of characters"

- strlen = returns the nr. of chars in a C string.
- strcpy = copies the characters from the source string to the dest. string.
- strcmp = compares two strings and returns:

↓
compare contents

{	0 if $a=b$
	<0 if $a < b$
	>0 if $a > b$

$\begin{matrix} = \\ < \\ > \end{matrix} \rightarrow$ compare memory addresses

- strcat = appends the characters from the source string to the end of the destination string.

! None of them check the size.
allocate memory.

7) Record - composite type ?

- $\rightarrow$ a collection of items of $\neq$ types.
- $\rightarrow$ group various data types into a structure
- $\rightarrow$ declared using struct.

typedef $\rightarrow$ introduce a new name for an existing type.


```
typedef struct
{
    int arr[100];
    int len;
} vector;
```

- vector can be used like any other variable.
- without "typedef", you need to write "struct" every time when using "vector". ?
- ex:

```
struct
{
    int arr[100];
    int len;
} vector
```

-
- ex:

- 8) scanf-s → not recommended (because it's not in the standard library)
- does not allow to put too big stuff
 - scanf can overflow, but scanf-s checks size and prevents that.

- 9) Variable = named location in memory. (label)

Memory: allocated according to the type of the variable

int → 4 bytes

char → 1 byte etc.

Types: tell the compiler how much memory to reserve for it
what operations ...

Value of the variable is undefined until initialization.

- 10) Constants

#define → preprocessor directive

const → keyword.

• INTEGER

ex: #define LENGTH 10

const int LENGTH = 10

• ENUMERATION CONSTANT

ex: enum colors {RED, YELLOW}

• FLOATING

ex: #define Pi 3.14

• STRING LITERAL

ex: const char * pc = "Hello" → lives in read-only memory
→ pc just stores its address.

char S[] = "Hello"

- creates array of 6 chars
- stores them directly in S.
- you own that memory ⇒ you can modify it.

first character of the string

11) Pointer = a variable whose ^{value} ~~name~~ is a memory location.
(can be the address of another variable)

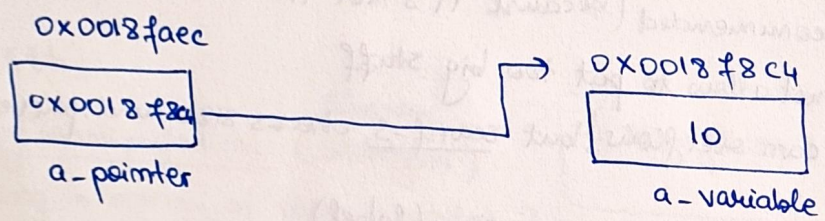
→ declaration

int *x; → asterisk
char *str;

→ operators

& = "address of" → take the ADDRESS of a variable.

* = "dereferencing operator" → take the VALUE at that address pointed to.



a-pointer = 0x0018f8c4 (value)

&a-pointer = 0x0018faec (address)

*a-pointer = 10 → "go in memory at the value contained in a-pointer (which is an address, btw) and at that address you will find a value → take that value".

a-variable = 10

&a-variable = 0x0018f8c4.

12) Statements

- ↳ empty statement
- ↳ compound statement
- ↳ conditional statement : if; if-else, else if, switch-case.
- ↳ loops: while, do-while, for.

13) Scope = the place where a variable was declared determines where it can be accessed from.

↳ function have their own scopes (variables defined inside the function will be visible only in the function)

Global var.

↳ the scope is the entire application.

14) PASS BY VALUE

`void byValue (int a)` → works on the copy of the parameter a

PASS BY REFERENCE

in C.

→ simulated with pointers

→ arrays!

→ pointers are passed by value.

`void byRef (int *a)`

(C)

`void byRef (int &a)`

(C++)

15) Test functions

`ASSERT`

`#include <assert.h>`

`void assert (int expression);`

if evaluation == 0 → message → stops execution.